

AI DevOps Assistant: AI-Assisted User Story Generator

Individual Project

By

Aye Khine Lwin

Student No: 140747

Professor - **Miguel Ângelo da Silva**

Subject - **Gestão de Projetos Ágeis (GP_03913)**

Date - 21/06/2026

Project Overview:

This AI-powered web solution was developed to automate the generation of standard Agile documentation, effectively minimizing the manual effort and time required from Product Owners while maintaining high-quality consistency during backlog management phases.

Objective:

To create an AI Web App that instantly converts rough, plain text into the Professional Agile format (User Story & Acceptance Criteria) to save time and ensure organization for software development teams when writing product backlogs.

This AI Web App was created to instantly convert rough text into the Professional Agile format (User Story & Acceptance Criteria).

1.Target Audience & Scope:

- **Target Audience:** Software Development Teams (**Product Managers, Scrum Masters, and Software Developers**) who struggle with manual backlog refinement and requirement gathering.
- **Project Scale/Size:** Designed for **small to medium-sized agile teams (approx. 5–15 members)** who need rapid AI-assisted drafting to reduce overhead and improve sprint planning efficiency.
- **Impact:** The solution **targets the 'J-curve' of AI adoption** by lowering the barrier for teams to integrate AI into their daily Scrum ceremonies.

2. Agile & Scrum Practices:

To manage this project, I adopted the Scrum framework:

- **Product Backlog:** I identified user stories (US01, US02, US03) and prioritized them based on project needs.
- **Kanban Board:** I used **GitHub Projects** which operated as a **Kanban board** to track tasks. Each task went through three states: **Todo**, **In Progress**, and **Done**.
- **Iterative Progress:** Instead of building everything at once, I built the UI first (MVP), then integrated AI, and finally the DevOps pipeline.

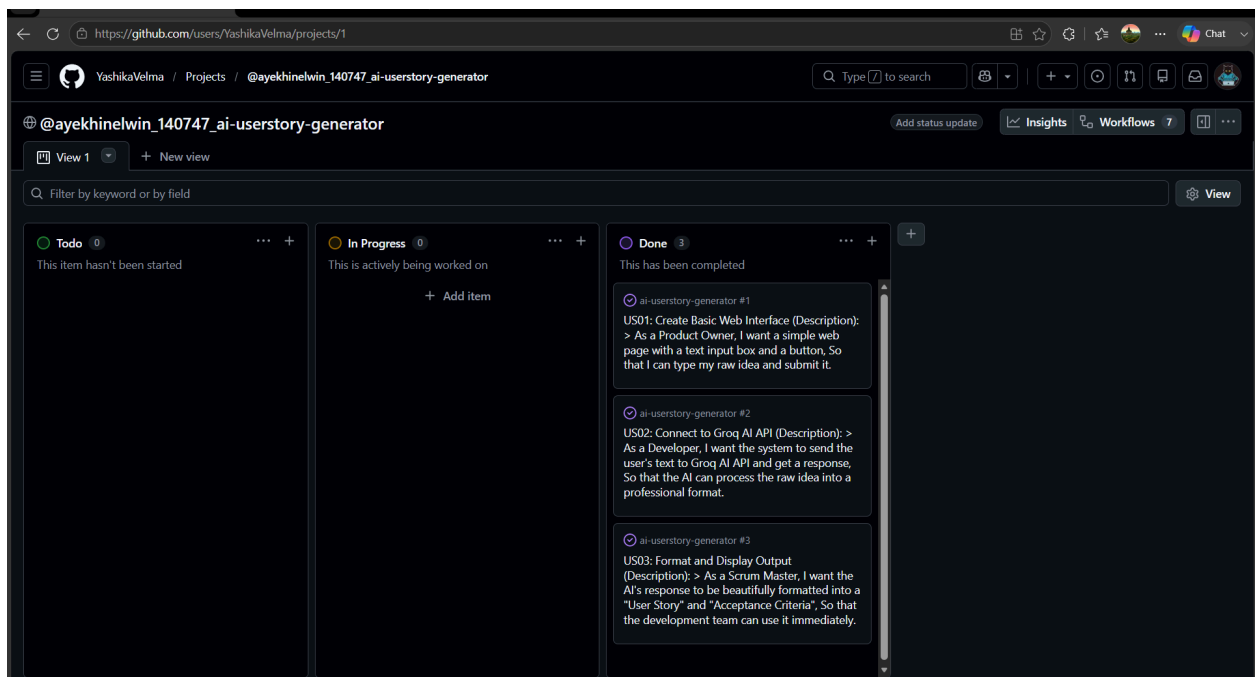


Fig 1: Github Project Board (Kanban Board) of AI-Assisted User Story Generator

3. Technical Architecture & Implementation:

The project follows a modern, scalable, and secure architecture designed for rapid development and deployment. The solution is divided into three primary components: the Frontend Interface, the AI Processing Engine, and the DevOps Pipeline.

3.1. System Architecture Overview

The system employs a client-server architectural pattern where the frontend serves as an interactive wrapper, and the backend utilizes a RESTful API to communicate with a Large Language Model (LLM).

3.2. Frontend & AI Integration

- Frontend Interface: I utilized Streamlit, a Python-based framework, to create a fast, interactive web interface. Streamlit was chosen for its capability to rapidly prototype and deploy data-driven web applications without complex HTML/CSS overhead.
- AI Engine: The core functionality is **powered by the Groq API**. I specifically implemented the **llama-3.3-70b-versatile** model due to its high inference speed and expert capability in domain-specific tasks such as business analysis and Agile coaching.
- Data Flow: When a user inputs a raw project requirement, the application sends a structured prompt to the Groq API. **The prompt is engineered to instruct the AI to act as an expert Business Analyst, ensuring that the response strictly follows the Agile User Story format ("As a..., I want..., So that...") along with comprehensive Acceptance Criteria.**

3.3. Security & Environment Management

Maintaining security in AI development is critical, especially regarding API credentials.

- Streamlit Secrets: I implemented the Streamlit Secrets feature to manage sensitive data. This allowed me to securely store the **GROQ_API_KEY** within the server configuration rather than hardcoding it into the source code.
- Configuration Access: Within the application logic, the key is accessed using **`st.secrets["GROQ_API_KEY"]`**, ensuring that credentials remain protected and are not exposed during version control commits. I manually put Groq API key through the Streamlit Web app setting from my workspace. Please see the screenshots below.

```

9     try:
10         api_key = st.secrets["GROQ_API_KEY"]
11     except Exception:
12         api_key = None

```

Fig2: Code snippet from [app.py](#) for API key

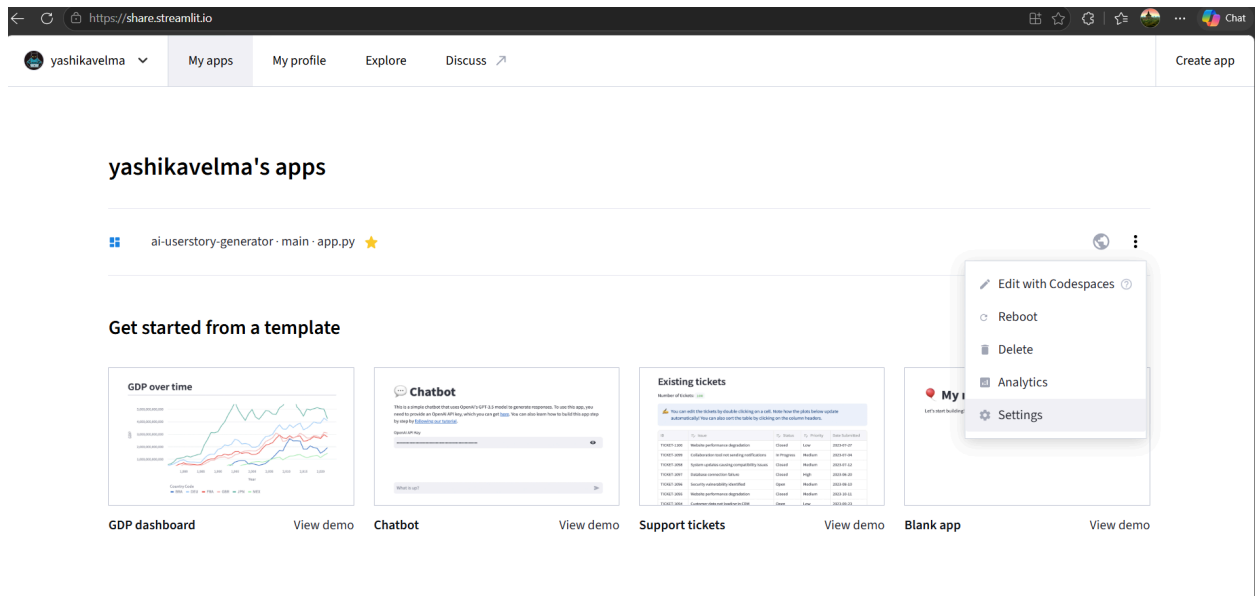


Fig3: App Setting in Streamlit Workplace

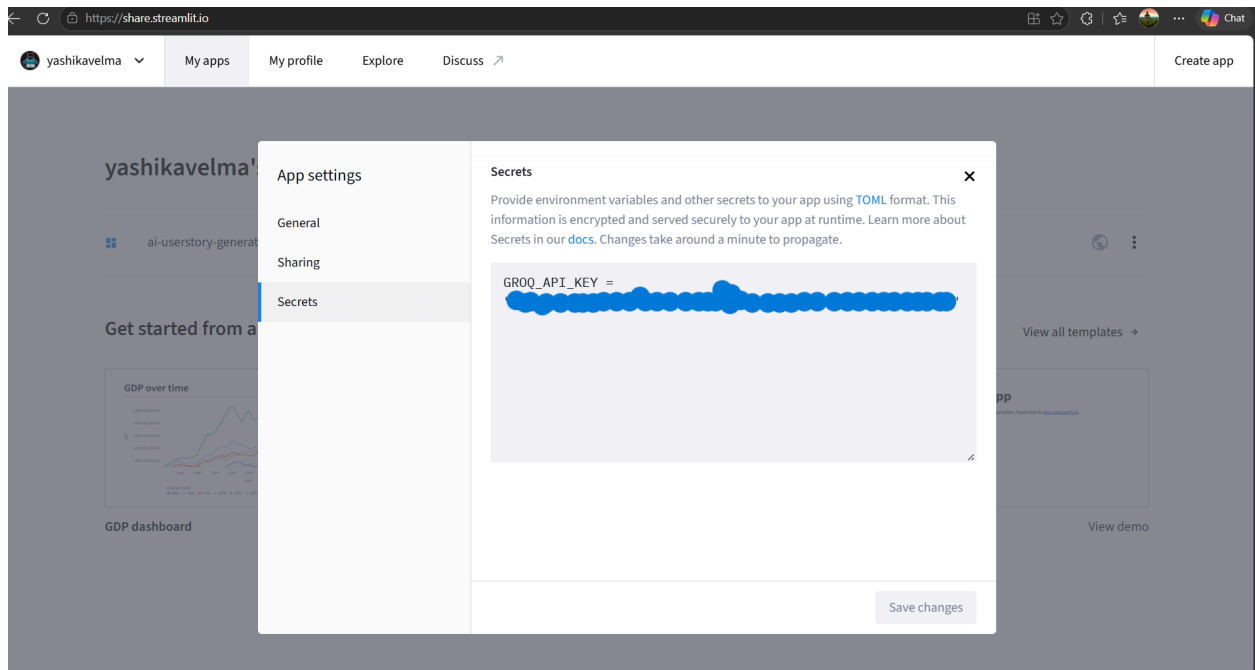


Fig4: Putting Groq API key in Streamlit Secrets So that user can use freely

3.4. DevOps Pipeline (CI/CD)

To maintain code quality and ensure the stability of the application, I integrated a Continuous Integration (CI) pipeline using GitHub Actions.

- Workflow Implementation: I configured a `.github/workflows/main.yml` file to automate code verification upon every `push` or `pull_request`.
- Quality Assurance: The pipeline automatically performs:
 1. Checkout: Retrieves the latest code from the Github repository.
 2. Environment Setup: Configures the Python 3.10 environment.
 3. Dependency Management: Installs necessary packages via `pip`.
 4. Syntax Verification: Executes `py_compile` on `app.py` to identify potential syntax errors before they reach production. *(Include Screenshot: The "Success" green checkmark from GitHub Actions showing the pipeline passing)*

Outcome - This ensure that no broken code is deployed.

(Reference Fig 5 images below: Successful GitHub Actions (Pipelines) execution logs and status indicators).

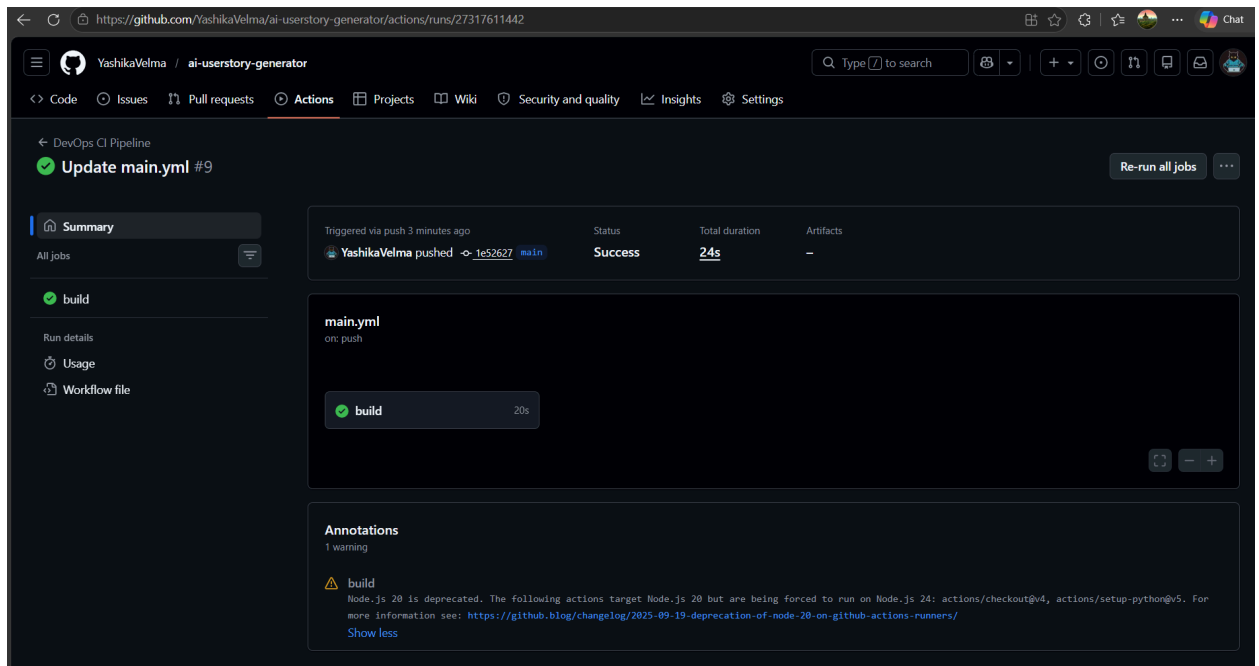


Fig 5.0: GitHub Actions (Continuous Integration (CI)) Build complete successfully.

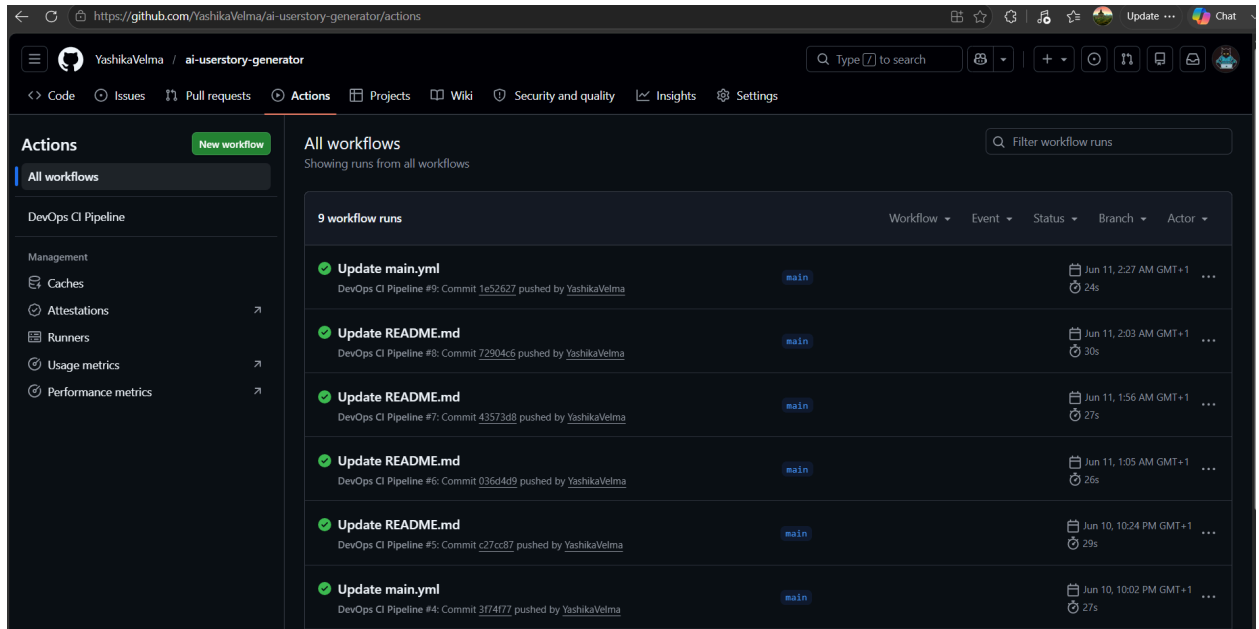


Fig 5.1: Successful GitHub Actions (Continuous Integration (CI)) execution logs and status indicators.

Which operational efficiency is driven by **GitHub Actions**, orchestrated through the `.github/workflows/main.yml` configuration.

4.Reflections & Core Competencies Gained:

- Security & Secret Management:** Addressed the complexities of safeguarding API credentials and operational keys.
 - Resolution:** Implemented `Streamlit Secrets` to ensure secure environment variable handling, moving away from high-risk hardcoded practices.
- Pipeline Integrity & Debugging:** Encountered significant DevOps bottlenecks, including automation failures and versioning conflicts.
 - Resolution:** Refined YAML configuration files and rectified environment dependencies, resulting in a more profound grasp of automated cloud deployment architectures. (Ref Fig 6 and 7)
- Key Synthesis:** This project illustrated that holistic development transcends basic programming; it necessitates rigorous process oversight, automated validation, and a security-first mindset.

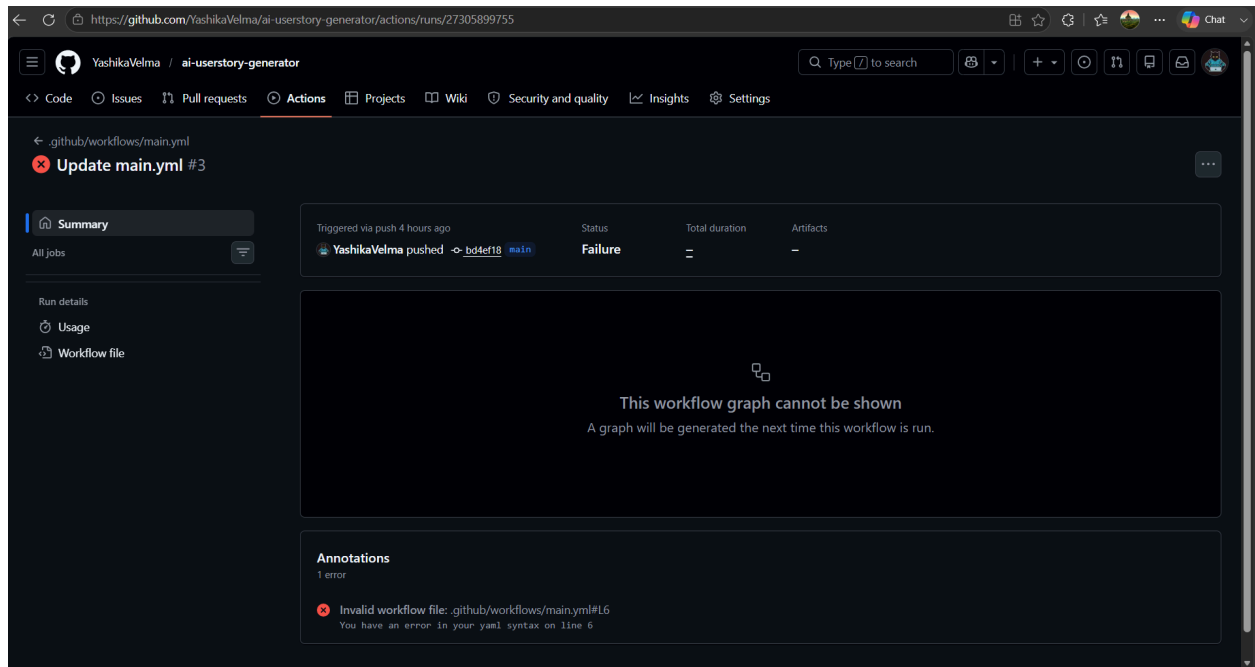


Fig 6: Error in **Continuous Integration (CI)** due to coding spaces and spelling.

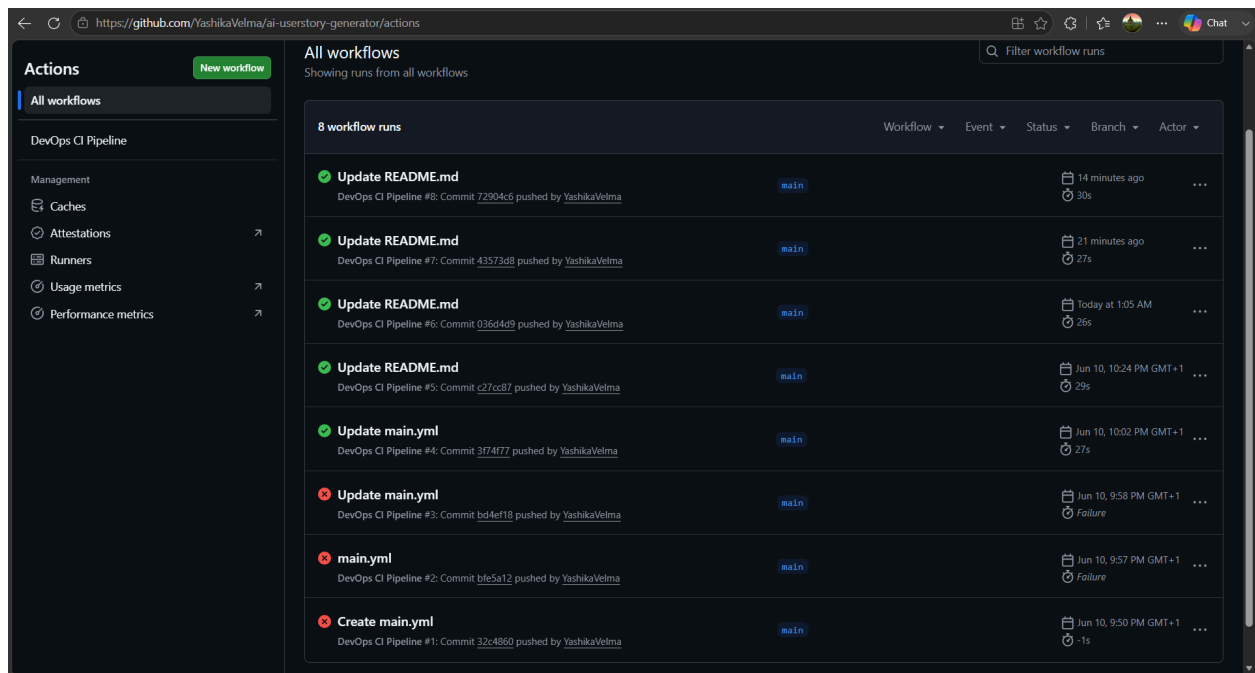


Fig 7: After repeated corrections of CI (main.yml) The Pipeline is fully operational

The images below are the real time User Story and acceptance criteria generated by the AI in the live web application”.

The Live Web Application Environment link: (Streamlit Cloud URL)
<https://ai-userstory-generator-fb24zshsngeyjedswgkpix.streamlit.app/>

Real time Outputs of AI-Assisted UserStory Generator :

- **User Story 1:** As a software engineering team lead, I want to leverage Artificial Intelligence (AI) to optimize and automate tasks within our Google Cloud-based systems, so that we can improve overall system performance, reduce manual errors, and increase development efficiency.

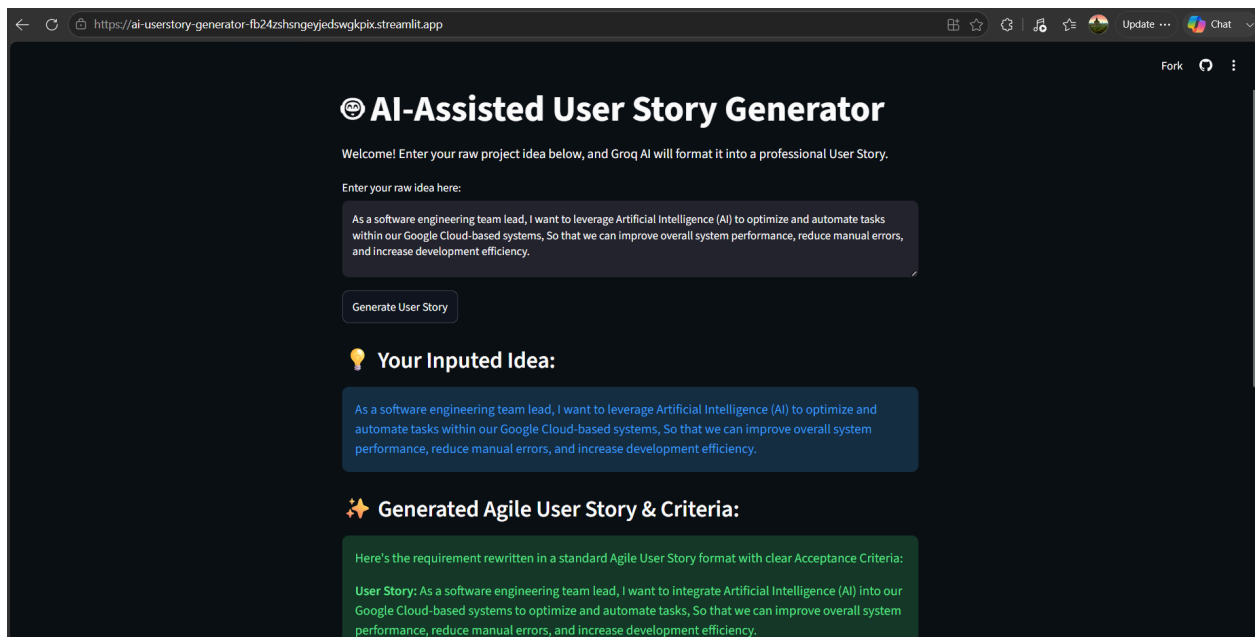


Fig 8: Inputting UserStory 1

- **Acceptance Criteria:**

Here's the requirement rewritten in a standard Agile User Story format with clear Acceptance Criteria:

1. AI Integration: The system successfully integrates with a chosen AI platform (e.g., Google Cloud AI Platform, TensorFlow) to automate tasks.

2. **Task Automation:** At least 3 key tasks within the Google Cloud-based systems are automated using AI, resulting in a minimum of 20% reduction in manual effort.
3. **System Performance Improvement:** The AI-optimized system demonstrates an average response time reduction of 15% and an average throughput increase of 10% compared to the previous manual process.
4. **Error Reduction:** The AI-automated tasks show a minimum of 30% reduction in manual errors compared to the previous manual process.
5. **Development Efficiency:** The software engineering team reports a minimum of 15% increase in development efficiency, measured by the number of features delivered per sprint, after implementing AI-automated tasks.
6. **Monitoring and Logging:** The system provides real-time monitoring and logging of AI-automated tasks, enabling the team to identify and address potential issues promptly.
7. **Security and Compliance:** The AI-integrated system adheres to Google Cloud security and compliance standards, ensuring the protection of sensitive data and maintaining regulatory compliance.

Definition of Done: The user story is considered complete when all acceptance criteria are met, and the software engineering team has successfully demonstrated the AI-optimized system's capabilities in a production-like environment.

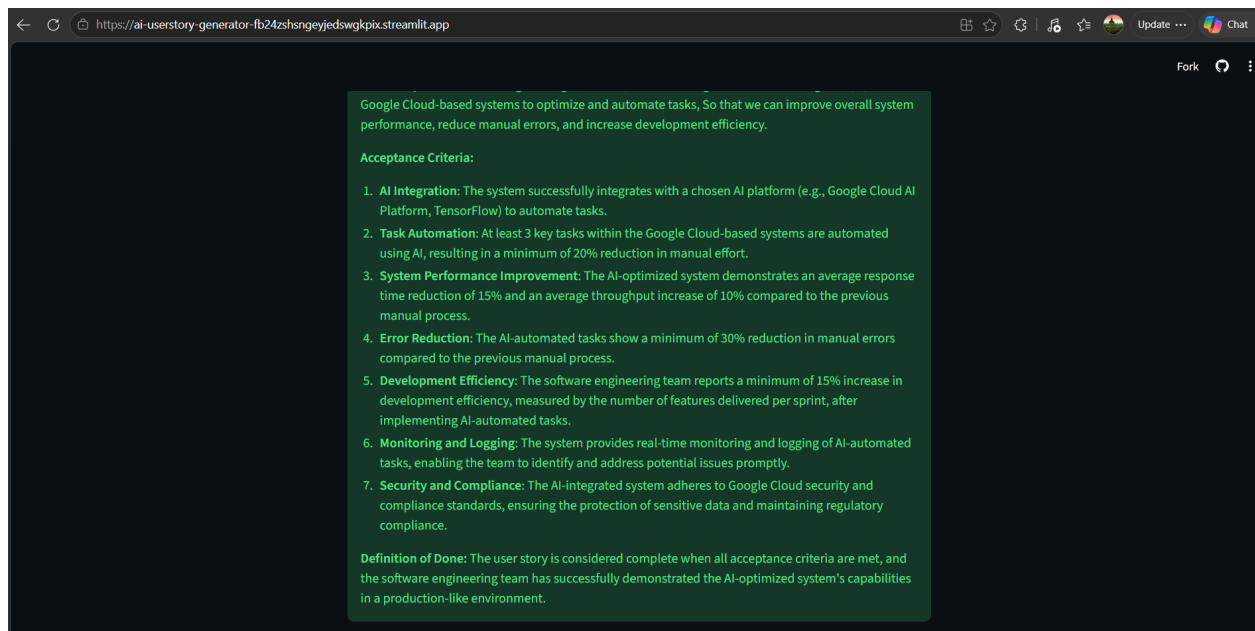


Fig 9: Acceptance Criteria of UserStory 1

5. Project Deployment & Resource Accessibility:

- **Live Application Environment:** The production-ready web solution is accessible via the following [Streamlit Cloud URL](#).
- **Operational Management:** Agile task orchestration and issue tracking were managed through the [GitHub Project Board](#), documenting the project lifecycle and sprint progress.

6. Conclusion & Final Assessment:

The initiative culminated in the delivery of a fully operational AI-driven DevOps utility. This endeavor highlights a sophisticated integration of artificial intelligence within Agile lifecycles, emphasizing that robust DevOps frameworks are indispensable for ensuring high-caliber code reliability and seamless delivery

References

1. Agile Methodologies & Scrum Framework:

- Silva, M. A. (2026). *Gestão de Projetos Ágeis / Agile Project Management* (Week 1, 2, 3, 5). ISCTE-IUL.

“The Agile Scrum methodology plays a key role in managing complex projects (Silva, 2026, Week 3)”.

2. DevOps & Engineering Practices:

- Silva, M. A. (2026). *Gestão de Projetos Ágeis / Agile Project Management* (Week 4, 6). ISCTE-IUL.
- Silva, M. A. (2026). *Productivity Gains of DevOps Adoption in an IT Team: A Case Study*. ISCTE-IUL.

3. AI in Software Development:

- Google Cloud. (2026). *The ROI of AI-Assisted Software Development*. DORA Research.

“Integrating artificial intelligence within software development environments significantly amplifies the operational efficiency of engineering ecosystems (Google Cloud, 2026)”.

- Silva, M. A. (2026). *Inteligência Artificial, Scrum e DevOps*. ISCTE-IUL.

4. Agile Principles & Coaching:

- Agile Alliance. (n.d.). *Agile Manifesto and Principles*. Retrieved from agilemanifesto.org